

Guide to Automated Morphometry: Quantitative Computer Vision for COF & Graphene Micrographs

Llinersy Uranga Piña, LPPI - Cergy-Paris Université

June 2026

1 Introduction: Morphometry vs. Manual Image Inspection

In solid-state structural chemistry and nanotechnology, processing microscopic data via manual feature picking introduces significant individual observer bias and lacks the statistical power required for quality control. Automated computer-vision morphometry treats micrographs (SEM, TEM, AFM) as quantitative physical matrices.

By applying digital filter pipelines, we convert varying electron beam intensity values into precise physical descriptors: grain diameter histograms, circularity factor arrays, and structural boundary lattice parameters.

2 The Core Image Analysis Pipeline Deconstructed

2.1 Phase 1: Background Standardization & Noise Mitigation

Raw microscopic imagery regularly contains significant variations in sample preparation thickness, spatial electron dispersion, and sample charging artifacts. This manifests as structural gradients (shadows) across the imaging frame.

Standard global binarization (fixed-level thresholding) fails here because a pore on a darker background shares the exact numerical grayscale value as a solid framework wall on a lighter background.

1. **Median Filtering vs. Gaussian Blurring:** To combat electronic detector flicker without wiping out crisp structural boundaries, we utilize a **Median Filter** over an operational spatial coordinate disk ($r = 1$ or 2 pixels). Unlike a Gaussian blur, which averages surrounding edges together, a median filter retains sharp interface drops while cutting high-frequency single-pixel noise spikes.
2. **Adaptive Local Thresholding:** Instead of implementing a flat, static intensity threshold across the global workspace, the mathematical threshold variable $T(x, y)$ evaluates localized windows ($N \times N$ pixel clusters):

$$T(x, y) = \mu_{\text{local}}(x, y) - \text{offset} \quad (1)$$

where $\mu_{\text{local}}(x, y)$ represents the localized mean intensity directly calculated inside the structural micro-neighborhood.

2.2 Phase 2: Morphological Cleaning

Following threshold tracking, the raw output contains unphysical structural fractures and minor single-pixel noise pockets. These are systematically removed using discrete morphological filters:

- **Minimum Size Filtration (remove_small_objects):** Eliminates isolated high-intensity clusters smaller than a specified pixel domain area (e.g., 20–50 px). This step prevents random noise from being misclassified as structural pores.
- **Binary Closing (binary_closing):** Performs a dilation step followed by an erosion operation utilizing a solid spatial disk element. This joins minor micro-cracks inside framework walls and fills pinholes without changing the overall size of the pores.

2.3 Phase 3: Marker-Controlled Watershed Segmentation

When individual pores are packed tightly together, their boundaries frequently bleed into one another in the image matrix. Standard object labeling algorithms treat these connected regions as a single giant "blob," skewing the final pore count. To isolate these regions, we use a **Marker-Controlled Watershed Segmentation** routine.

1. **Euclidean Distance Transform (EDT):** Every foreground pixel within the binary mask is mapped to a scalar value corresponding directly to its straight-line distance to the nearest background wall edge:

$$D(x, y) = \min_{(x_b, y_b) \in \text{Background}} \sqrt{(x - x_b)^2 + (y - y_b)^2} \quad (2)$$

The local centers of the pores appear as peak maximum coordinates in this calculated topography.

2. **Local Peak Selection (peak_local_max):** We identify these geometric center points by applying a coordinate restriction parameter (`min_distance`), which establishes a minimum allowable pixel distance between adjacent peaks. These points serve as the initial "seed points" or markers for the final segmentation step.
3. **Topographical Flooding:** The inverse distance landscape ($-D(x, y)$) is mathematically "flooded" with water starting from each unique marker coordinate. Where rising fluid fronts originating from separate markers collide, the algorithm draws an exact physical dividing boundary line. This isolates individual pore spaces cleanly.

3 Extracting Solid-State Metrics from Pixel Arrays

3.1 Lattice Spacing Computations via Spatial Distance Arrays

For highly ordered systems such as Covalent Organic Frameworks (COFs), calculating the average distance between neighboring pore center coordinates gives us an instant estimation of the structural lattice parameter.

Given a coordinate matrix of extracted pore peak positions $\mathbf{P}_i = (x_i, y_i)$, an $N \times N$ spatial distance matrix is constructed. By filtering for the closest valid neighbor vectors, we calculate the empirical mean spacing value:

$$a_{\text{lattice}} = \frac{1}{N} \sum_{i=1}^N \left(\frac{1}{k} \sum_{j \in \text{Nearest}_k} \|\mathbf{P}_i - \mathbf{P}_j\| \right) \quad (3)$$

For standard hexagonal arrangements, evaluating the top 4 nearest valid vectors per node filters out edge clipping effects while providing a robust estimation of structural lattice regularity.

3.2 Porosity Fractional Ratio Calculation

The 2D physical fractional porosity (ϕ_{2D}) is calculated by tracking the ratio between the total pixel area assigned to labeled pore void spaces and the overall image dimension boundaries:

$$\phi_{2D} = \frac{\sum \text{Area}_{\text{Pores}}}{\text{Height}_{\text{Pixels}} \times \text{Width}_{\text{Pixels}}} \times 100\% \quad (4)$$

While ϕ_{2D} does not substitute for a volumetric 3D BET gas adsorption measurement, it tracks changes in surface structural evolution in real-time during sample synthesis adjustments.

4 Guidelines for Processing Your Own Experimental Data

To apply this automated processing notebook to your personal research images (e.g., changing from the simulated template to a custom raw file like `images.jpeg`), you must update the core file attributes inside the loading block.

```
1 # --- Student Workspace Action Item ---
2 # To process your custom research image, update the configuration below
3 :
4 my_file = "images.jpeg"      # Ensure file name matches your file
5                               # exactly
6 pixel_width = 310            # Enter the true horizontal pixel count
7                               # Enter the true vertical pixel count
8 pixel_height = 163
9
10 # Execute the updated morphometry function
11 experimental_data = analyze_experimental_micrograph(
12     filepath=my_file,
13     target_width=pixel_width,
14     target_height=pixel_height
15 )
```

Listing 1: User Calibration Parameters for Micrograph Morphometry

4.1 Troubleshooting Structural Segmentation Errors

- **Over-Segmentation (Too many tiny false pores found):** This is typically caused by high-frequency noise from low crystallinity or electronic beam jitter. To fix this, increase the `sigma` parameter in your pre-filtering step, or increase the `min_distance` value within your peak extraction configuration.
- **Under-Segmentation (Merged pores/fewer objects found):** This occurs when low image contrast washes out fine framework features. To fix this, decrease the adaptive threshold `offset` value or widen your morphological processing `block_size` to capture more subtle background intensity transitions.